

Managed ArgoCD Scalability

Speaker Notes

Slide 1: Managed ArgoCD Scalability

Today I will be discussing how Managed ArgoCD can evolve to support broader platform adoption while maintaining a scalable and reliable operating model.

I'll start with a brief overview of ArgoCD and its key capabilities. Then I'll walk through how Managed ArgoCD is being used today, the opportunities we see for broader adoption, the challenges with the current architecture, and finally the proposed architectural direction.

The goal of this presentation is not to review a detailed design, but rather to establish the problem statement, quantify the impact, and validate the proposed direction before moving into detailed design.

Slide 2: What is ArgoCD?

Before discussing Managed ArgoCD, I want to briefly cover what ArgoCD is and why platform teams use it.

ArgoCD is a GitOps control plane for Kubernetes. It continuously monitors the desired state defined in Git and reconciles it with the actual state running in Kubernetes clusters.

Beyond application deployment, ArgoCD provides capabilities that are particularly useful for platform teams.

It supports multi-cluster application management, application health monitoring, deployment visibility, fine-grained RBAC, and ApplicationSets for fleet-wide deployments.

The important takeaway is that ArgoCD provides a standardized operational model for deploying, managing, and monitoring workloads across Kubernetes environments.

Slide 3: Managed ArgoCD Adoption Today

Before discussing future opportunities, I want to highlight how Managed ArgoCD is being used today.

At present, Managed ArgoCD is already supporting both platform services and application teams.

On the platform side, services such as Managed Spark Streaming and Managed PostgreSQL use ArgoCD to manage operators, controllers, and supporting infrastructure components.

On the application side, several teams operating dedicated single-tenant clusters use Managed ArgoCD to deploy and manage their workloads. Examples include API Proxy and P13.

Through these use cases, Managed ArgoCD has already demonstrated several benefits.

These include standardized deployment workflows, improved operational visibility, automated reconciliation, and strong access control through ArgoCD RBAC.

The key takeaway is that Managed ArgoCD is already providing value in production and has established itself as a reliable GitOps platform for current consumers.

Slide 4: Why Expand Managed ArgoCD?

While Managed ArgoCD has already proven successful for current consumers, the opportunity extends beyond the teams that are using it today.

Across the organization, there are multiple platform services that manage namespaces, clusters, controllers, operators, and other shared infrastructure components.

These services often face similar operational challenges.

They need to deploy common components, manage infrastructure across many environments, maintain operational consistency, and provide visibility to their users.

ArgoCD already provides capabilities that address many of these needs.

The opportunity is to position Managed ArgoCD not as a solution for a handful of services, but as a reusable platform capability that can support a broader set of platform offerings.

Slide 5: Problem Statement & Quantification

So far, we've looked at what ArgoCD is, how Managed ArgoCD is being used today, and where we see opportunities for broader adoption.

The next question is whether the current architecture can support that adoption.

Today, all Managed ArgoCD consumers rely on a single ArgoCD control plane.

At the current scale, we're managing approximately 250 clusters and more than 3,000 applications across production and non-production environments.

To support this growth, the Managed ArgoCD deployment has already been scaled to a 14-node cluster.

While vertical scaling has helped us accommodate current demand, it doesn't fundamentally change the architecture. All workloads, applications, and platform consumers still depend on the same control plane.

Looking ahead, we see opportunities for Managed ArgoCD to be adopted by additional platform services that manage namespaces, clusters, controllers, and shared infrastructure components.

If Managed ArgoCD can provide a scalable and reliable operating model, these services can leverage the same GitOps capabilities that are already delivering value for our current consumers.

This broader adoption has the potential to increase the Managed ArgoCD footprint to thousands of clusters and hundreds of thousands of applications. These are estimated values intended to illustrate the order of magnitude of growth rather than an exact forecast.

The important point is that scalability itself is not the goal.

Scalability is the enabler that allows Managed ArgoCD to evolve from a solution used by a few services into a reusable platform capability that can support a much broader set of consumers.

The challenge we're trying to solve is ensuring that future growth and adoption are not constrained by a single control plane architecture.

Slide 6: Solution Requirements & Value Proposition

Now that we've established the problem, the next step is to define what success looks like.

The goal is not simply to make ArgoCD bigger.

The goal is to create an architecture that allows Managed ArgoCD to be adopted more broadly while maintaining the qualities that have made it successful today.

First, the architecture must scale beyond the current consumer base and support future platform adoption without depending on a single control plane.

Second, it must improve reliability by reducing shared dependencies and limiting the blast radius of operational issues.

Third, we want to maintain operational simplicity. Any architectural changes should not significantly increase operational burden for platform teams.

Finally, we want to preserve the existing Managed ArgoCD experience. Consumers should continue to benefit from native ArgoCD capabilities such as RBAC, application visibility, health monitoring, and GitOps workflows.

The value of solving this problem is that Managed ArgoCD can evolve from a solution used by a handful of teams into a reusable platform capability that can serve a much broader set of platform services.

These requirements form the basis for evaluating potential architectural approaches.

Slide 7: Proposed Architecture Direction

Based on the requirements we just discussed, the proposed direction is to move away from a single control plane model and adopt a distributed control plane architecture.

The idea is to introduce multiple ArgoCD control planes while continuing to provide a unified Managed ArgoCD experience to consumers.

Instead of concentrating all workloads behind a single ArgoCD instance, workloads would be distributed across multiple ArgoCD control planes.

This directly addresses the requirements we established earlier.

For scalability, capacity can grow horizontally by adding additional ArgoCD instances.

For reliability, operational issues are isolated to a subset of workloads rather than impacting all consumers.

For operational flexibility, ArgoCD instances can be managed and upgraded independently.

Most importantly, consumers continue interacting with Managed ArgoCD as a single platform service. Native ArgoCD capabilities such as RBAC, application visibility, health monitoring, and GitOps workflows remain unchanged.

At this stage, the goal is not to finalize implementation details. The goal is to validate the architectural direction and determine whether a distributed control plane model is the right approach for enabling broader platform adoption.

Slide 8: Next Steps & Feedback Requested

This presentation is intended to establish the problem statement, quantify the impact, and propose a high-level architectural direction.

The next step is to convert this direction into a detailed architecture proposal.

Areas that require further design include workload placement, routing, capacity planning, operational processes, and reliability considerations.

At this stage, I am not seeking approval of a detailed implementation.

The primary goal is to validate three things.

First, whether the problem statement resonates and aligns with the broader platform vision.

Second, whether a distributed control plane model is the right architectural direction.

And third, whether there are additional considerations, constraints, or existing patterns that should influence the design.

If there is alignment on the direction, the next checkpoint will focus on the detailed architecture and implementation approach.